RL HW5 Report

Teoman Kaman

March 2026

1 Q-learning vs SARSA

Q1

(a)

It is considered an off-policy method because it updates Q-values using the Q-values of the next state s and the greedy action a ($\max Q(s,a)$). It estimates the return for state–action pairs assuming a greedy policy were followed, despite the fact that it may actually be following an ϵ-greedy policy.

The Q-learning update rule is

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big(R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\Big)$$

which explicitly uses the greedy action value $\max_a Q(S_{t+1}, a)$.

(b)

Yes. If action selection is purely greedy, then SARSA will always choose the action with maximum Q-value in the next state. Thus it makes it the same as in Q-learning ($\max Q(s,a)$).

The SARSA update is

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\Big)$$

but if the policy is greedy then

$$A_{t+1} = \arg\max_a Q(S_{t+1}, a)$$

so the update target becomes the same as in Q-learning.

So both algorithms use the same value in the update and will also select the same actions.

2 TD Learning vs Monte Carlo

Q2

Monte Carlo can perform better than TD in cases where the reward is only given at the end of an episode. In this case there are no intermediate rewards, so TD updates may not provide useful information during the episode.

TD learning updates values using the bootstrap target

$$R_{t+1} + \gamma V(S_{t+1})$$

while Monte Carlo waits until the end of the episode and uses the full return

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots$$

Monte Carlo waits until the end and uses the full return G, which correctly reflects the final outcome. Therefore MC can perform better than TD in episodic tasks with clear final rewards, such as chess or other board games.

3 Random Walk Experiment

3.1 3.1

The TD(0) algorithm was implemented for the random walk example. Each episode starts from the middle state and the agent moves left or right with equal probability until a terminal state is reached.

The TD update rule is

$$V(S_t) \leftarrow V(S_t) + \alpha\big(R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\big)$$

After many episodes the estimated values approach the true values.

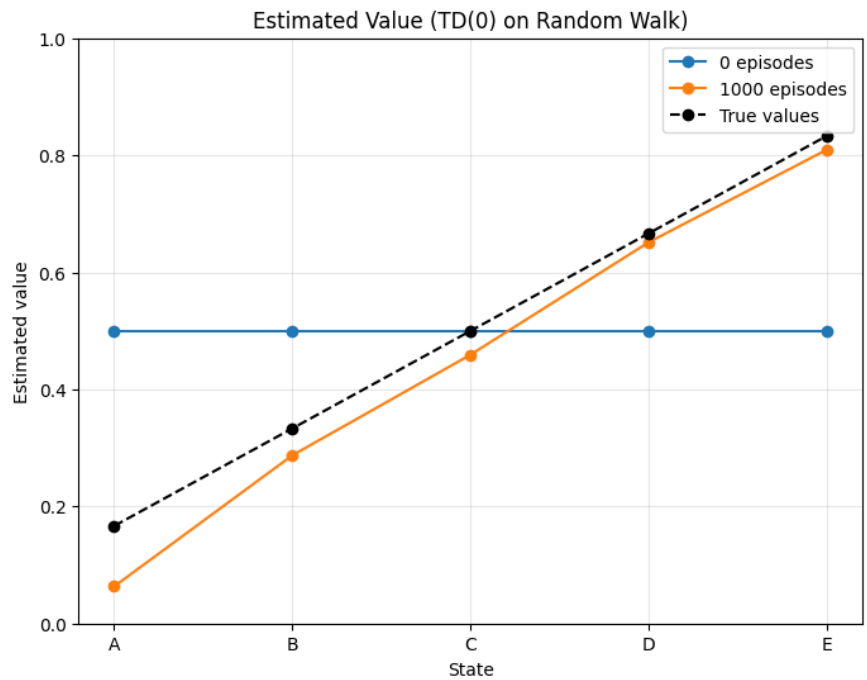

Figure 1: Estimated values after training using TD(0).

3.2 (B) Effect of Step Size

(i)

No, the overall conclusion is unlikely to change. Using a wider range of α values may slightly change the curves, but the general behavior should remain similar. TD(0) benefits from bootstrapping, which allows it to update value estimates earlier during an episode. In contrast, Monte Carlo must wait until the end of the episode to update. Because this advantage is structural, it is unlikely that a different fixed value of α would make Monte Carlo perform significantly better than TD on this task.

(ii)

This could happen if α was large so that the step size of updates becomes larger. The TD update is

$$V(S_t) \leftarrow V(S_t) + \alpha\,(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

If α is large, the updates become bigger. At the beginning it can reduce the error rapidly, but later the estimates may start to overshoot and fluctuate, which makes the error larger.

This does not always occur. It can depend on initialization. In the random walk example all states were initialized to $V(s) = 0.5$, which is already close to the true values. With a large α, TD may move the estimates away from this good starting point and cause instability. With different initialization or a smaller α, this situation might not happen.

4 Windy Gridworld

4.1 (a)

The Windy Gridworld environment was implemented based on Example 6.5 from the RL textbook.

The grid size is 7×10. Wind strength depends on the column and pushes the agent upward after each action.

The start state and goal state follow the configuration described in the book.

4.2 (b)

In this plot the axes are different from previous plots. The x-axis shows time steps and the y-axis shows the number of completed episodes. This is because episodes finish at irregular time intervals. Some policies reach the goal faster, so they complete more episodes within the same number of time steps. Therefore this plot measures how quickly each algorithm learns to finish episodes. Monte Carlo completes fewer episodes early on since it must wait for full episodes before updating, while TD methods update at every step

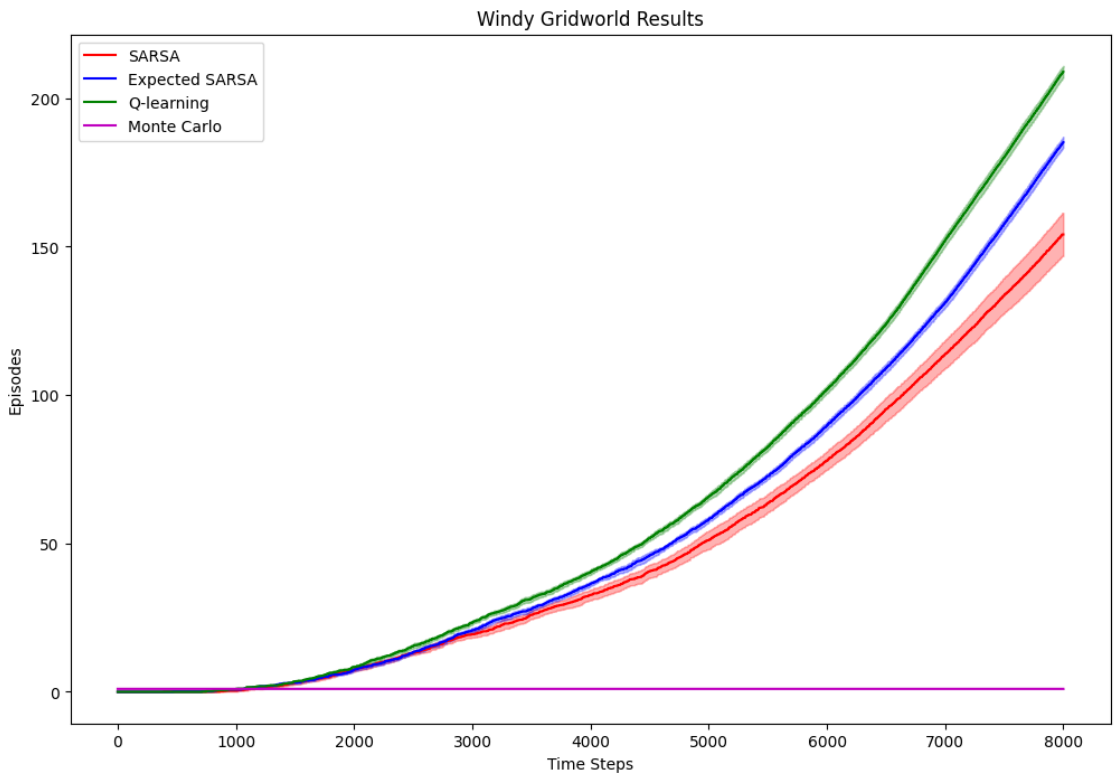

Figure 2: Comparison of SARSA, Expected SARSA, and Q-learning.

From the plot we see that Q-learning completes more episodes over time, which means it learns faster. Expected SARSA performs between SARSA and Q-learning.

4.3 (c) King's Moves

Allowing King's moves improves performance because the agent can move diagonally and reach the goal using shorter paths. From the plot we can see that the number of completed episodes after 8000 time steps increases significantly compared to the basic gridworld. This shows that additional movement options allow the agent to learn more efficient trajectories.

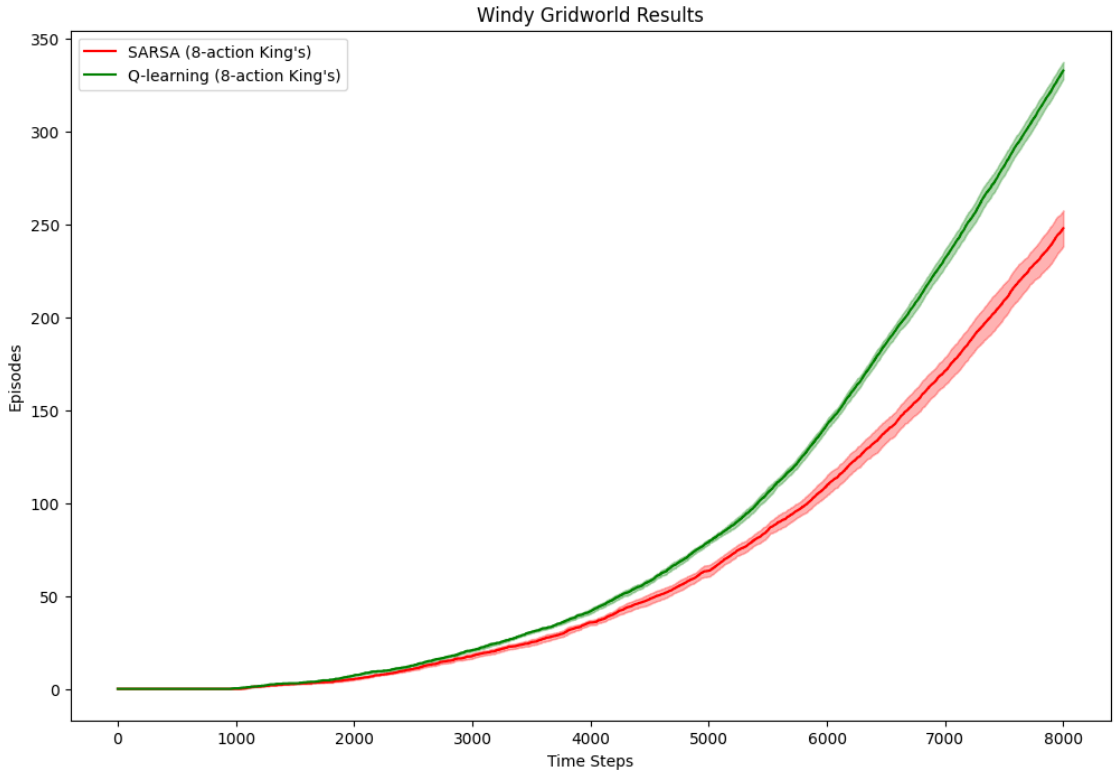

Figure 3: Windy Gridworld with King's moves.

Allowing diagonal movement improves performance because shorter paths become possible.

4.4 (c) Nine Actions

Adding a ninth action allows the agent to stay in place while the wind still affects the position. This can sometimes help the agent use the wind more effectively. The plots show that performance improves compared to the four-action case, although the improvement is smaller than with King's moves. This is because the additional action mainly helps in specific states rather than providing shorter movement paths.

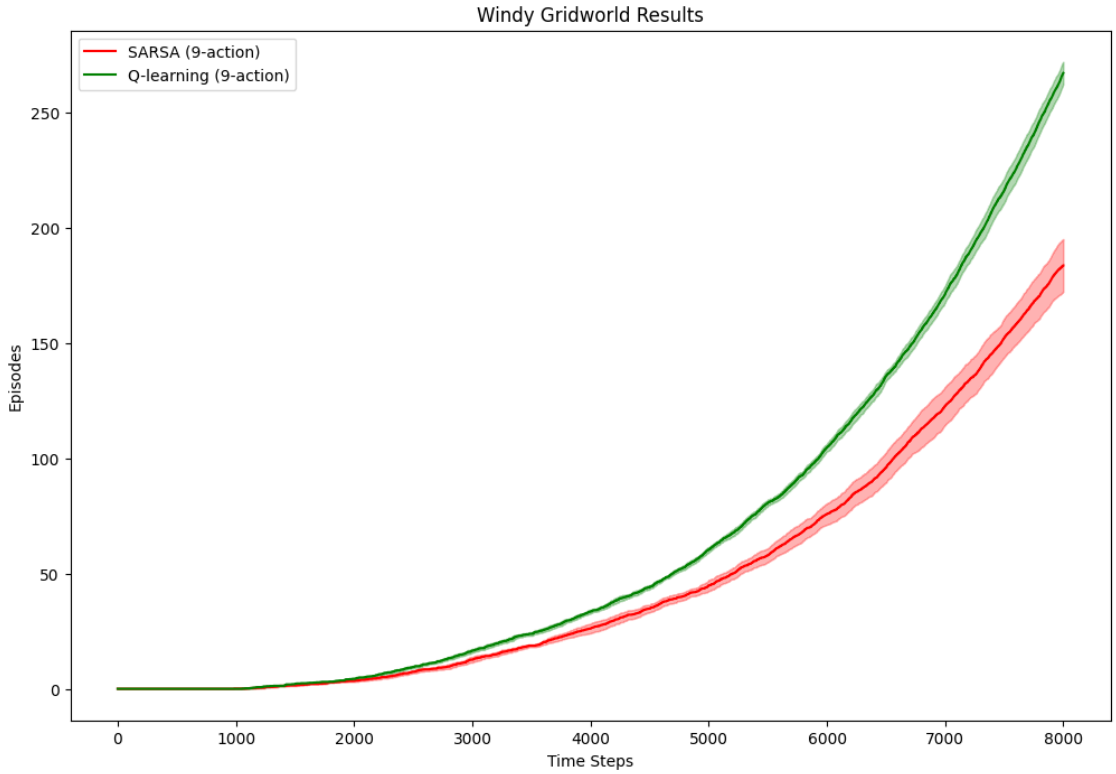

Figure 4: Windy Gridworld with 9 actions.

This action sometimes helps the agent use the wind to reach better states.

4.5 4(D)

The wind was also changed to stochastic wind, where the wind strength may vary.

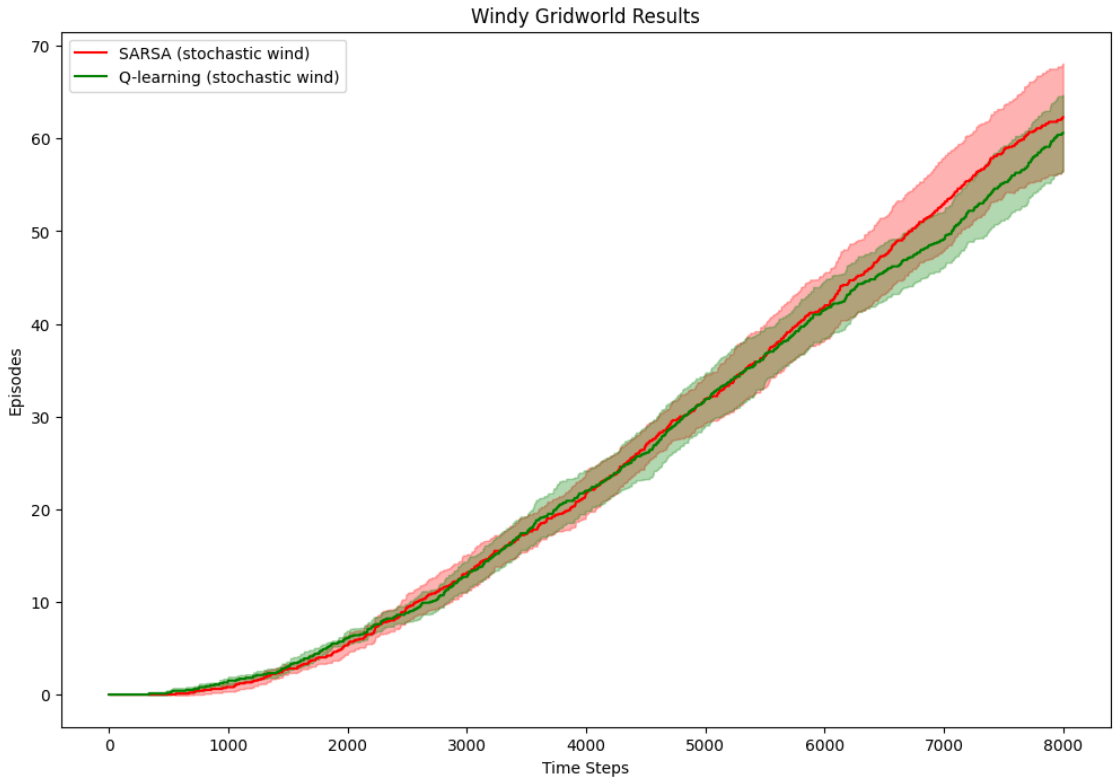

Figure 5: Windy Gridworld with stochastic wind.

Stochastic wind introduces uncertainty, which makes the learning process harder and increases variance.

5 Bias–Variance Analysis

In the final experiment we compare TD(0) and Monte Carlo learning targets.

For TD learning the target is

$$R + V(S')$$

For Monte Carlo the target is the full return

$$G$$

Histograms were generated for different numbers of training episodes.

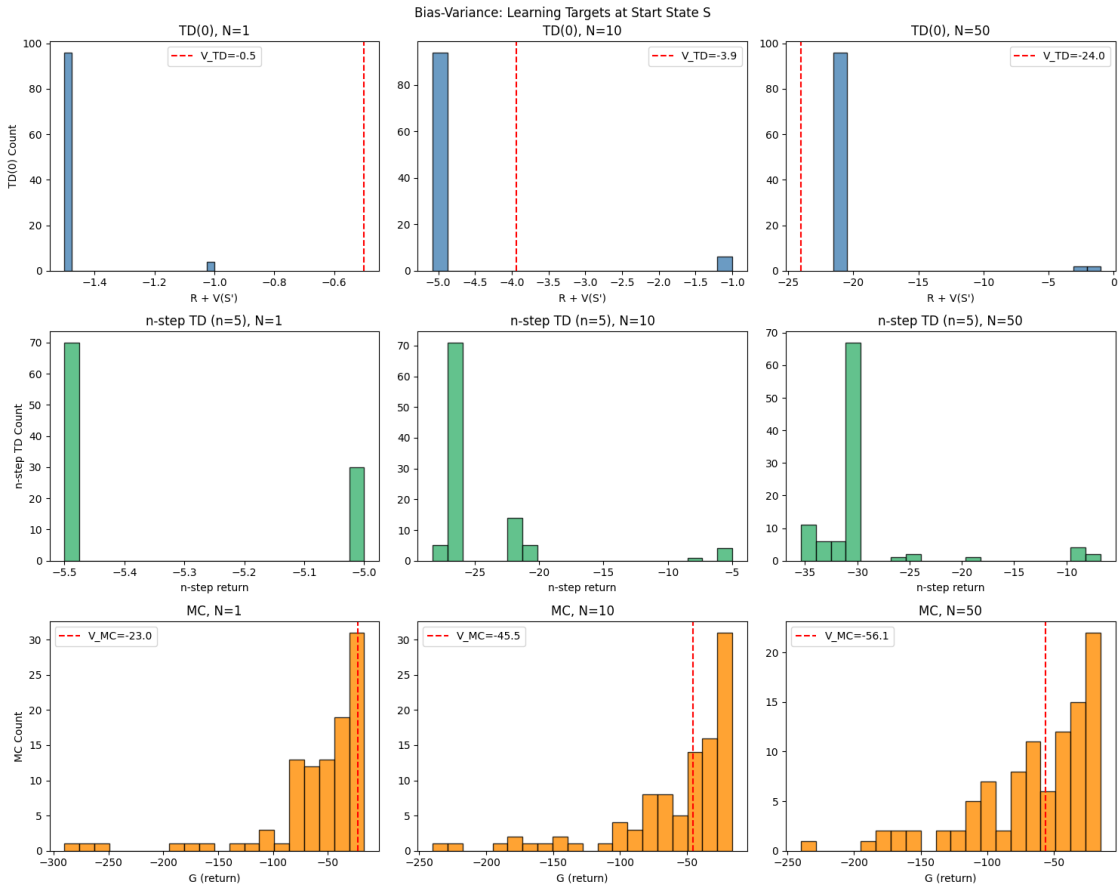

Figure 6: Distribution of TD and Monte Carlo targets.

The plots show that Monte Carlo targets have much larger spread, which indicates higher variance.

TD targets are more concentrated because they use bootstrapping. However, TD learning may introduce bias because it depends on the current value estimates.

5 (b)

TD(0) exhibits very low variance targets (a concentrated histogram) because it bootstraps immediately after one step using the current estimate $V(S')$. n-step TD has intermediate variance since it combines a few real rewards with a bootstrap estimate.

Monte Carlo has the highest variance because the full return G depends on every step of a potentially long episode. As N increases, the TD(0) estimate $V(S)$ becomes more negative and moves closer to the true value, which reduces bias.

Monte Carlo values also shift as more data is collected, but the variance of the targets remains large regardless of N. This illustrates the bias–variance trade-off between the methods.

With very little training ($N = 1$), TD(0) is strongly biased because the estimate $V(S)$ is still close to its initialization. In contrast, Monte Carlo already reflects the true variability of returns, even though its variance is high.

Q5(c)

In the control setting the policy π is not fixed. The agent improves the policy after learning new Q-values. Because of this, the distribution of learning targets changes over time.

For TD(0), the target $R + V(S')$ changes as the value estimates and the policy change. This makes the targets non-stationary, but the variance of individual TD targets remains relatively low.

For Monte Carlo, the variance usually becomes smaller over time. As the policy improves, the agent reaches the goal more efficiently and episodes become shorter and more consistent. This makes the return G less spread out.

Overall, during control the variance of Monte Carlo targets can decrease as the policy converges, but early in training the bias–variance trade-off is still strong because the policy is still poor.